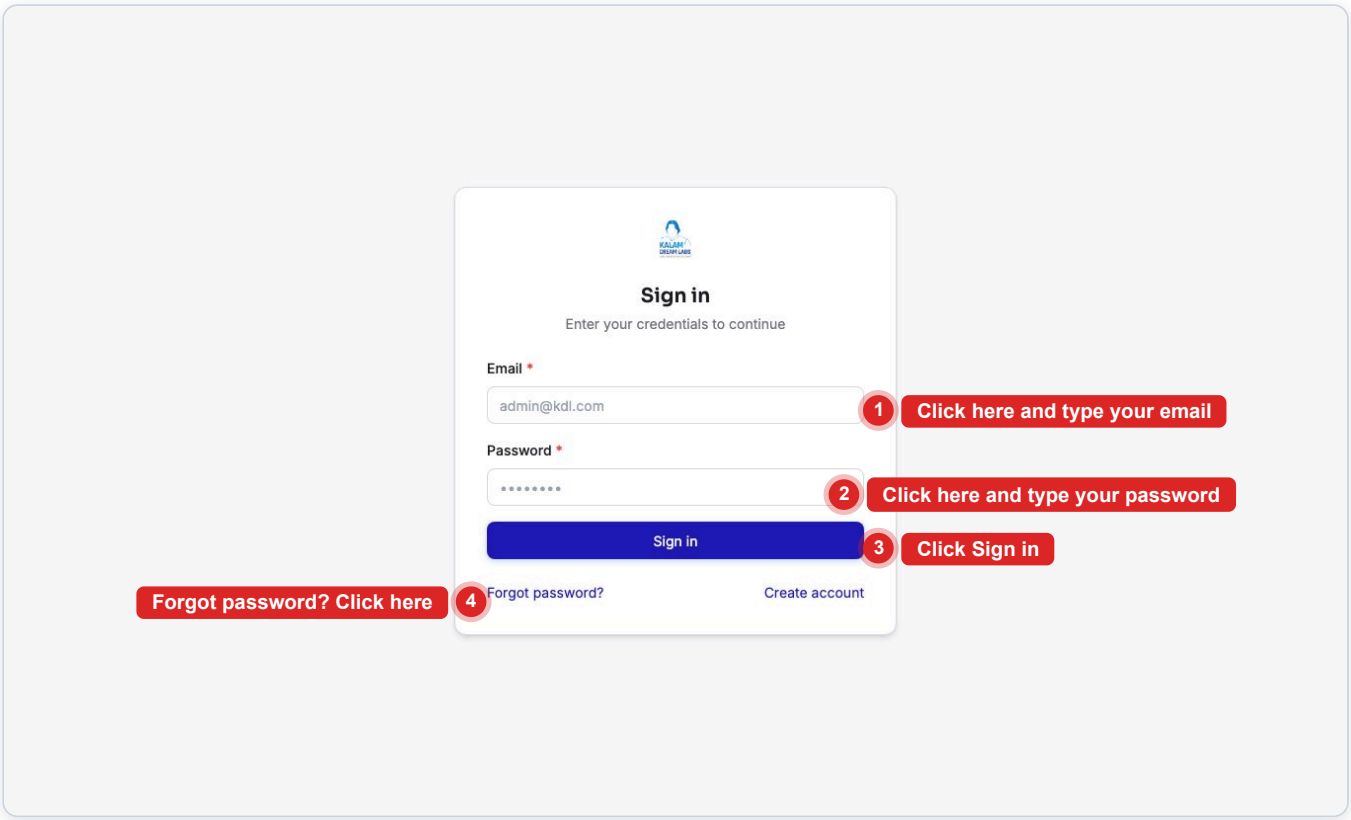
Step 1. Sign in

> KDL OS must already be installed and running. If not, follow the **Installation** guide first (Admin > Documents > Installation).

Open **http://localhost/login**. Email `admin@kdl.com`, password `kdl-dev-seed-password`. KDL OS asks you to choose a new password on first sign-in.

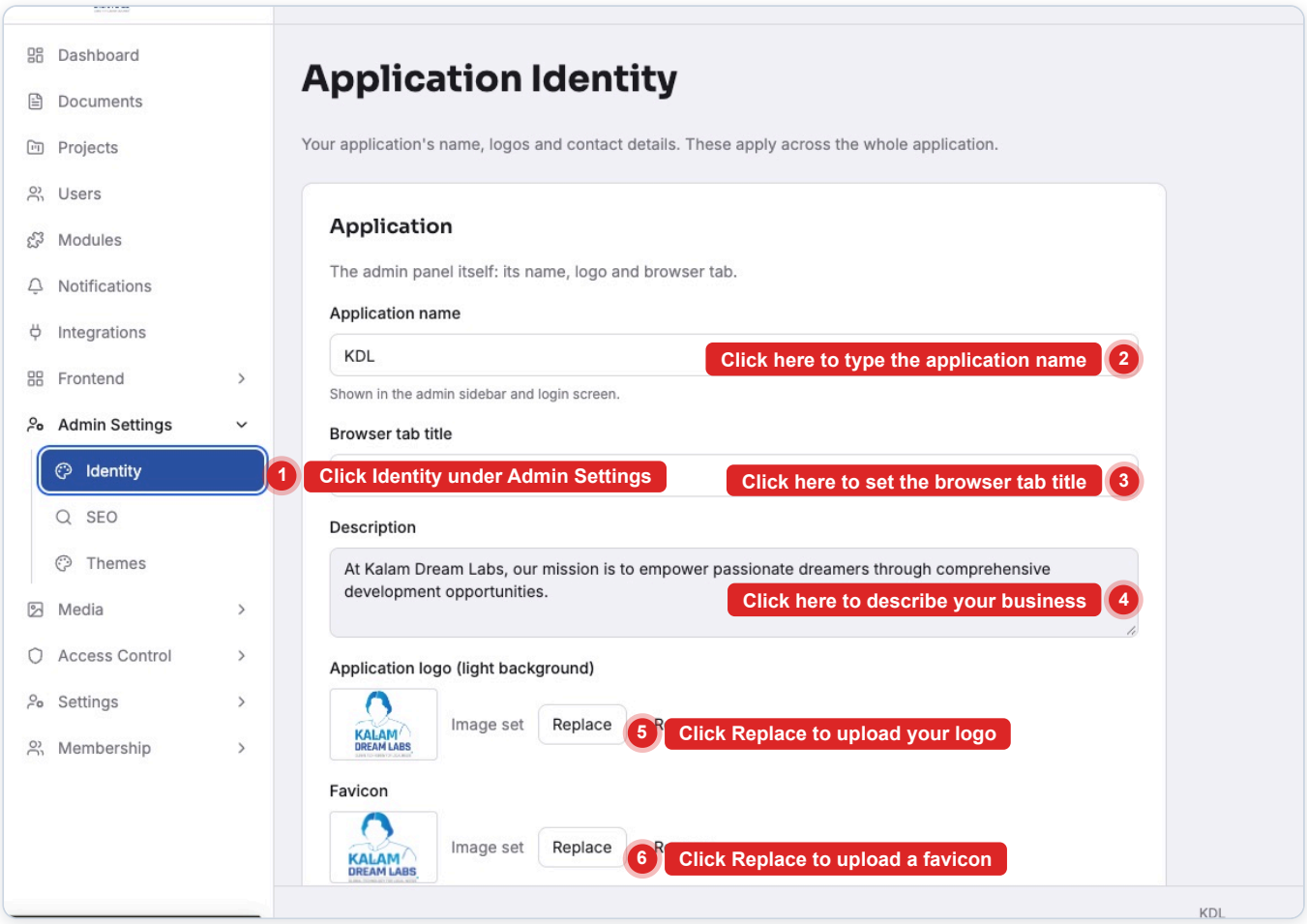


The sign-in screen

> Forgot the password or locked out? `docker compose exec backend node scripts/reset-admin-password.js --email admin@kdl.com --password 'Admin@123'` (at least 8 characters), then sign in with it. 429 or "Account locked" while testing: wait a minute or `docker compose restart backend`.

Step 2. Open Application Identity

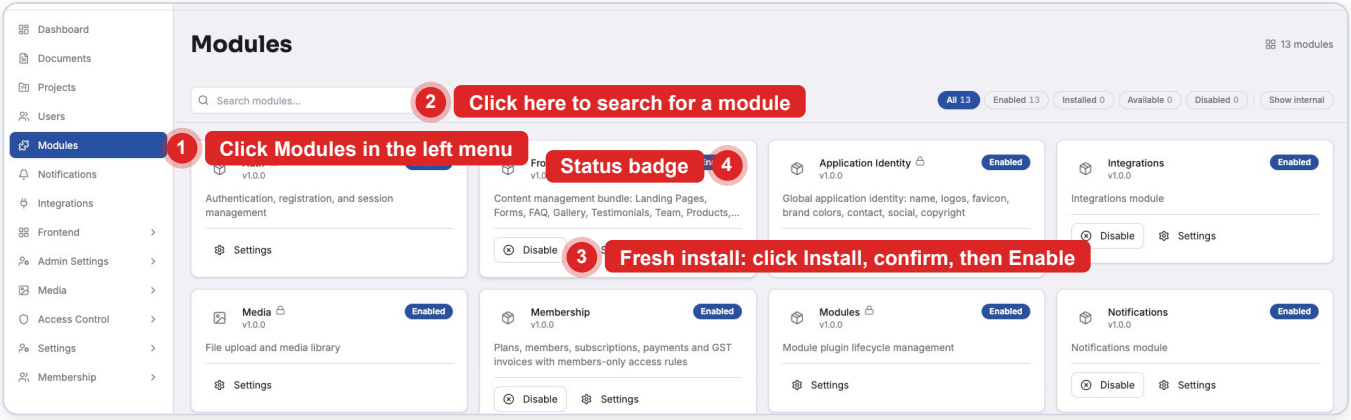
After you sign in, open **Admin Settings > Identity** in the left menu. This is where the application name, logo and favicon are set.



Admin Settings > Identity

Step 3. Open Modules, install and enable Frontend

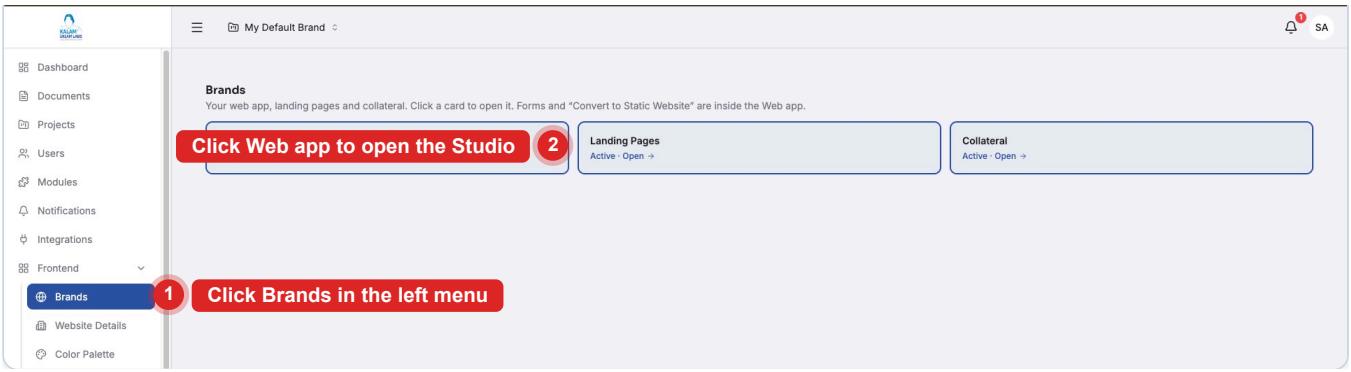
Click **Modules** in the left menu. Find the **Frontend** card (use the search box if you need to). On a fresh install the card shows **Install**: click it and confirm, then click **Enable** and confirm. The badge changes to **Enabled**.



Modules: the Frontend card

Step 4. Open the Studio

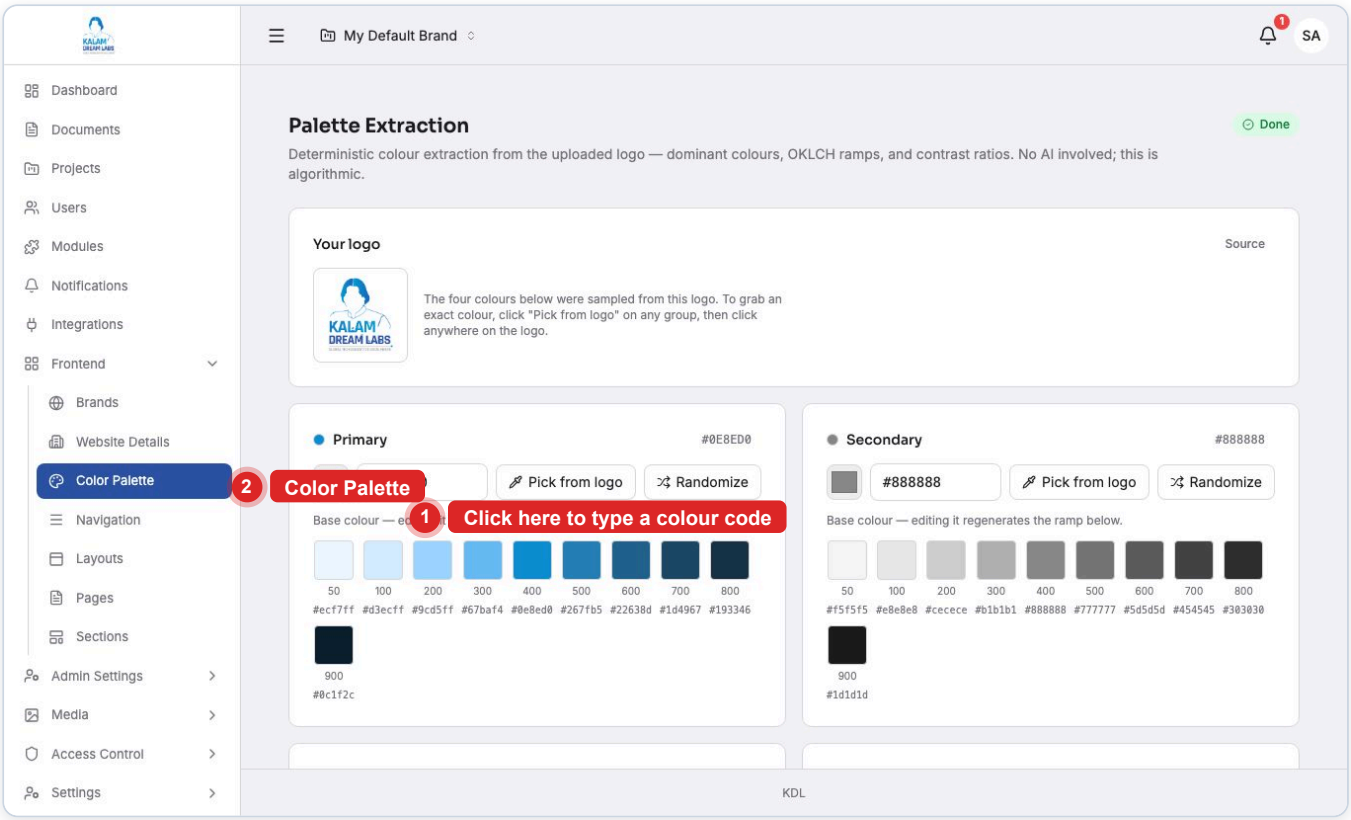
Click **Frontend > Brands** in the left menu, then click the **Web app** card. This opens the Studio for your website. The first time, click **Start Studio run**. Tabs along the top: Overview, Color Palette and Website.



Brands: open the Web app

Step 5. Color Palette

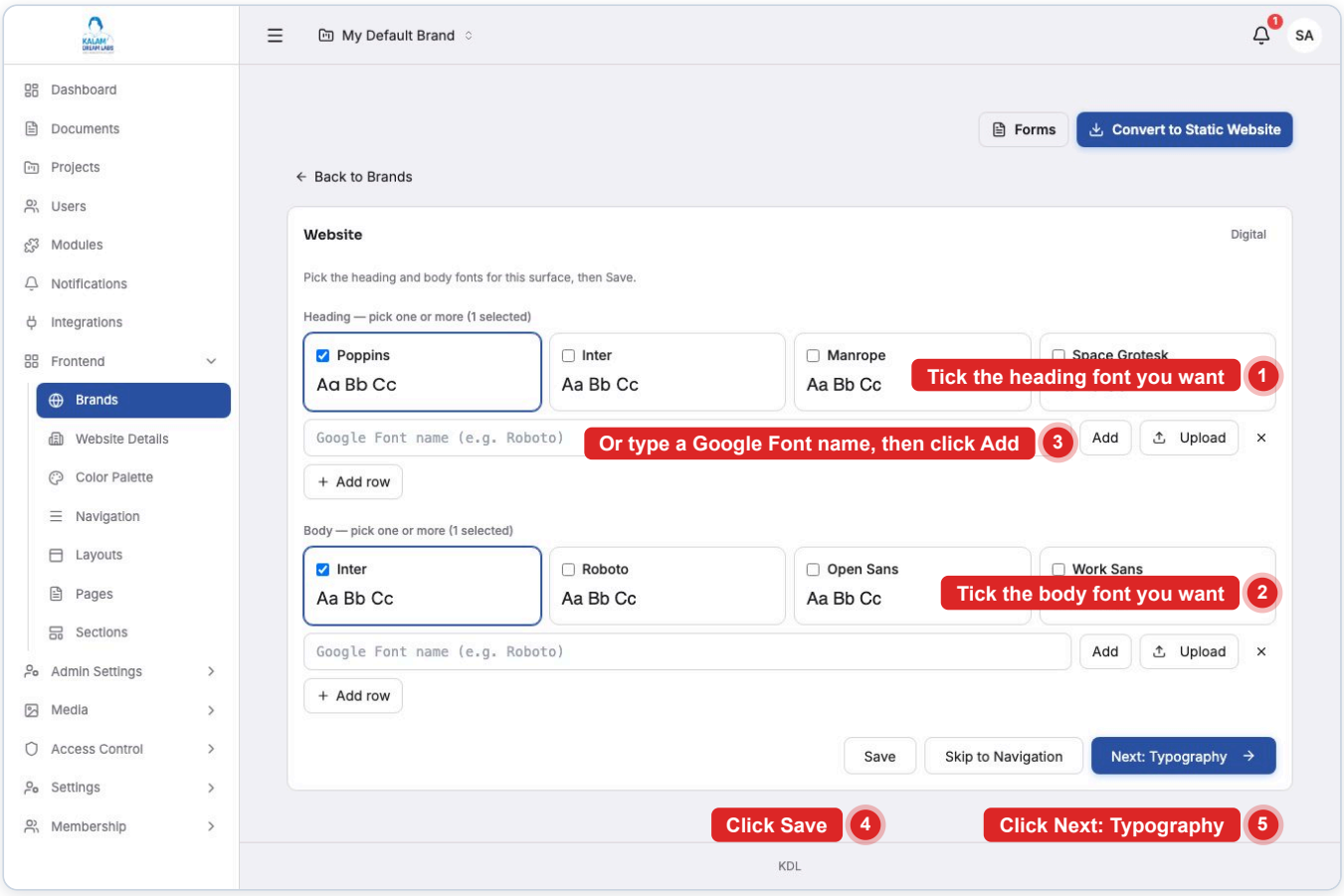
Four brand colours are taken from your logo: Primary, Secondary, Tertiary and Quaternary. Keep them, or click a colour box and type a new code, click **Pick from logo** or **Randomize**. Then click **Submit palette**. The colours become the website's design tokens.



Studio: Color Palette

Step 6. Typography: choose the fonts

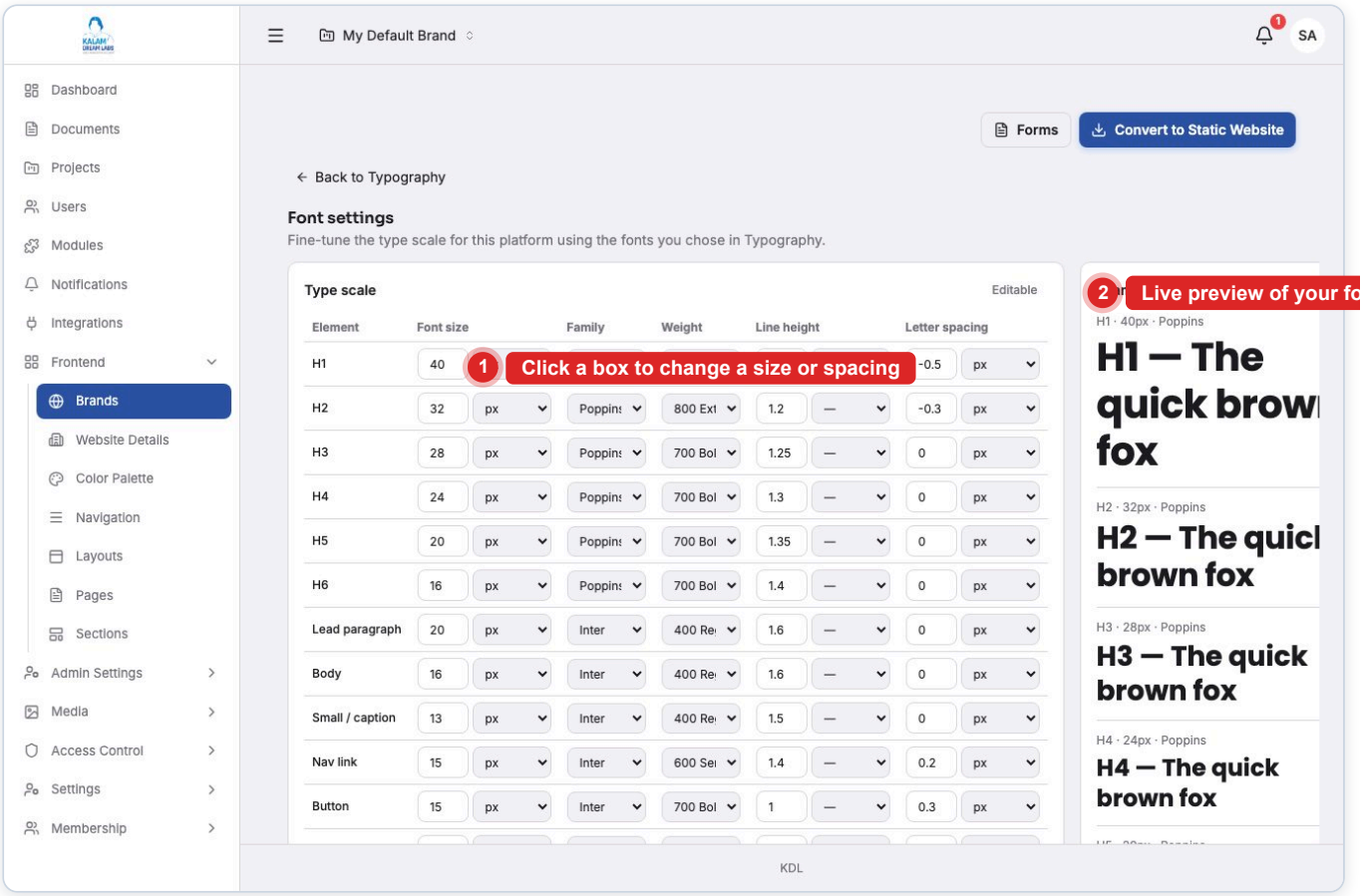
The Website stage opens on **Typography**. Under **Heading** tick the font for titles, and under **Body** tick the font for normal text. To use another font, type its Google Font name (for example Roboto) and click **Add**, or click **Upload** to add your own font file. Click **Save**, then click **Next: Typography** to continue.



Website stage: Typography

Step 7. Font settings

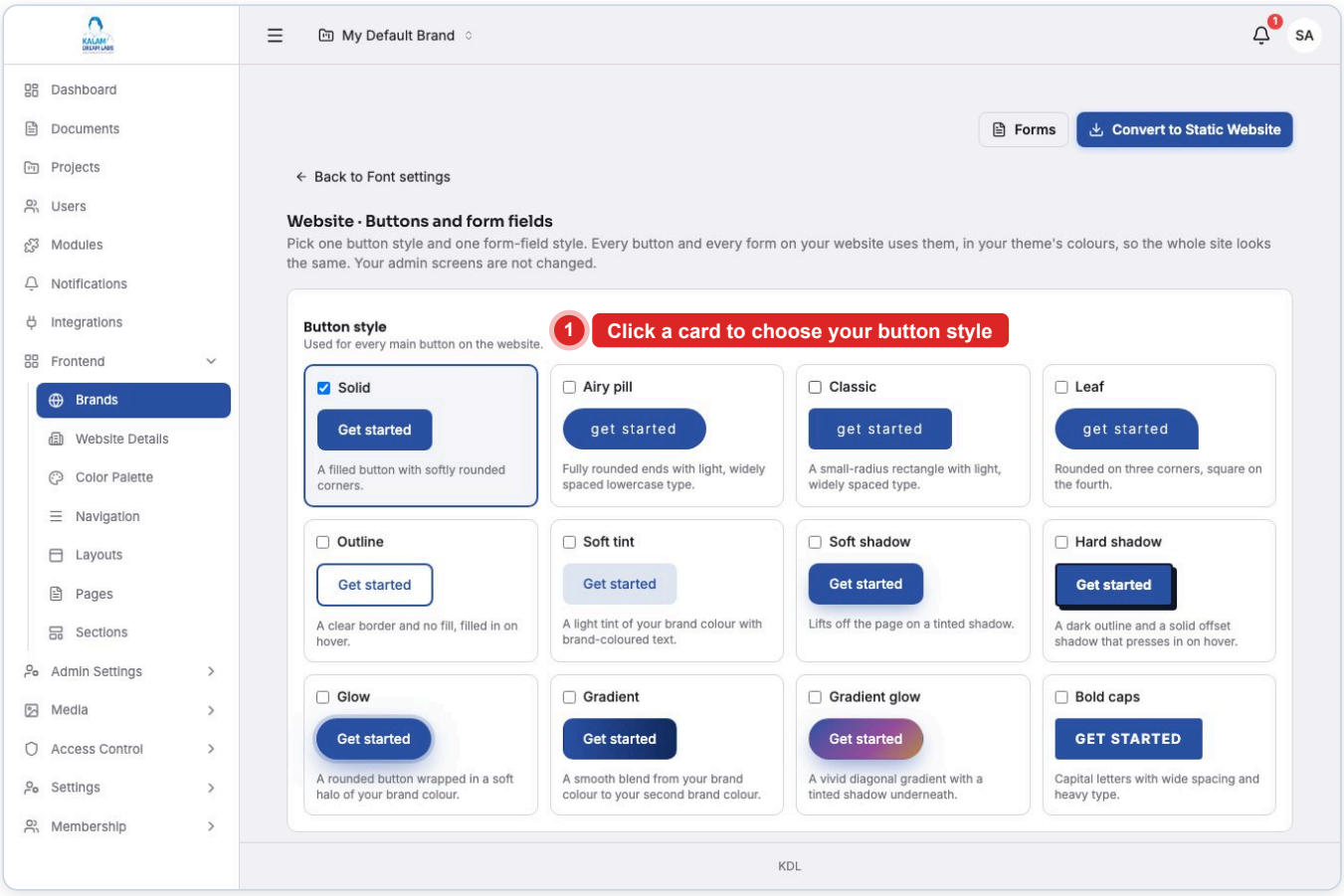
Fine-tune the type scale: for each element (H1 to H6, Lead paragraph, Body, Small, Nav link, Button) change the font size, family, weight, line height and letter spacing. The **Examples** panel on the right shows the result live. The defaults are a good start. Scroll down and click **Next: Buttons and form fields**.



Website stage: Font settings

Step 8. Buttons and form fields

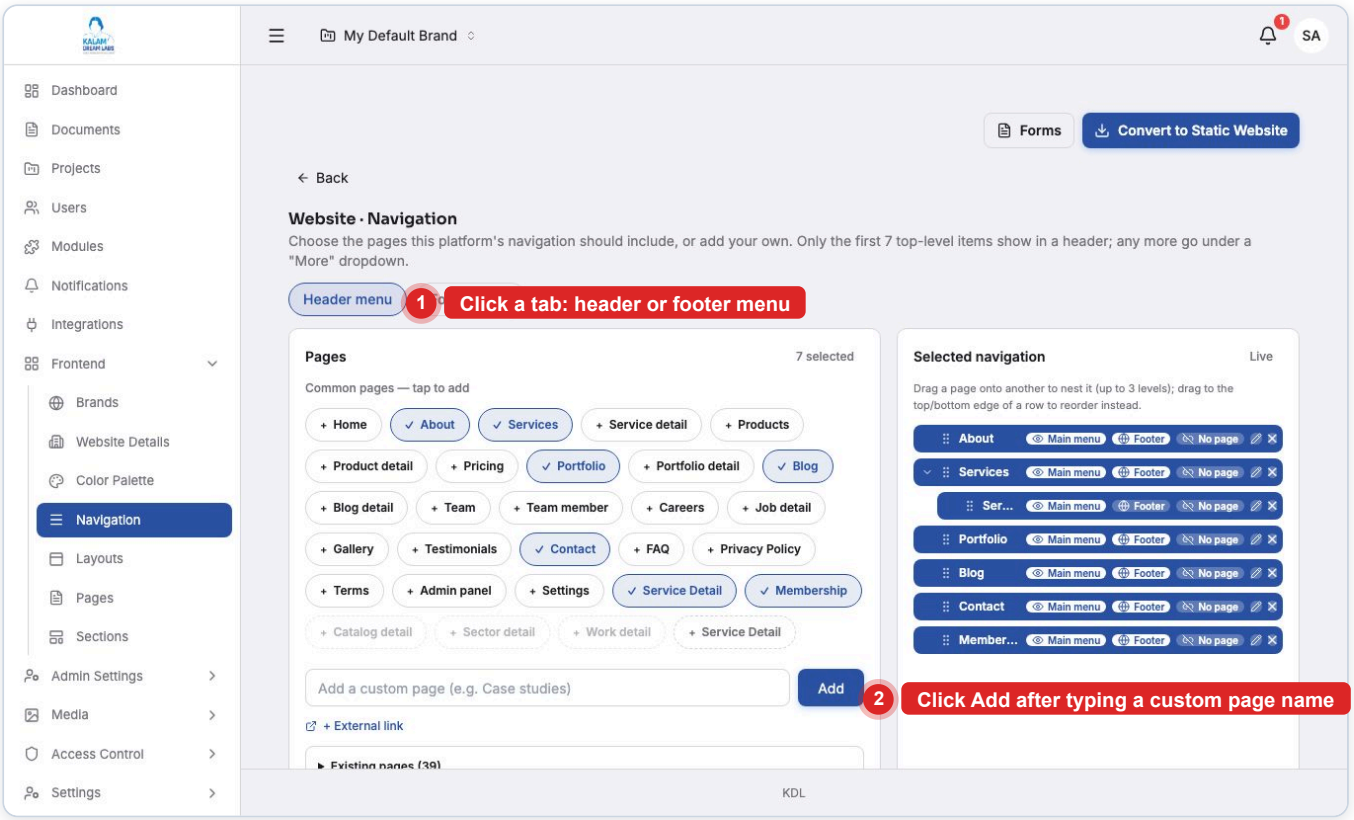
Click one card under **Button style** (Solid, Airy pill, Classic, Leaf, Outline, Soft tint and more), then scroll down and click one card under **Form field style**. Every button and every form on your website uses them, in your theme's colours. The admin screens are not changed. Click **Next** to continue to Navigation.



Website stage: Button style

Step 9. Choose the menu pages

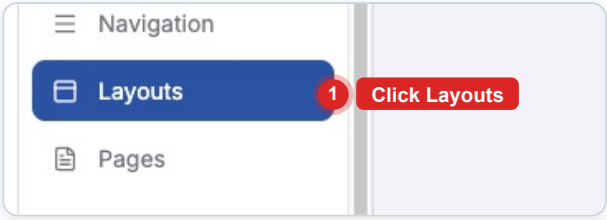
Open **Frontend > Navigation**. Click the page names you want in the menu (About, Services, Portfolio, Blog, Contact and so on), or type a custom page and click **Add**. Use the **Header menu** and **Footer menu** tabs. One page is created for every menu item, plus Home.



Frontend > Navigation

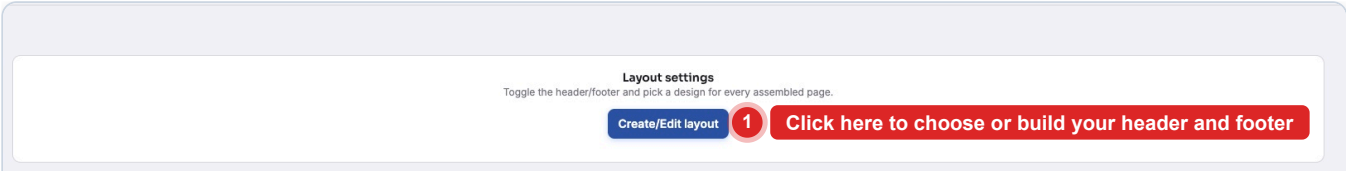
Step 10. Layouts: header and footer

Click **Layouts** in the left menu (under **Frontend**). The page shows **Layout settings**: the header and footer used on every page of your website.



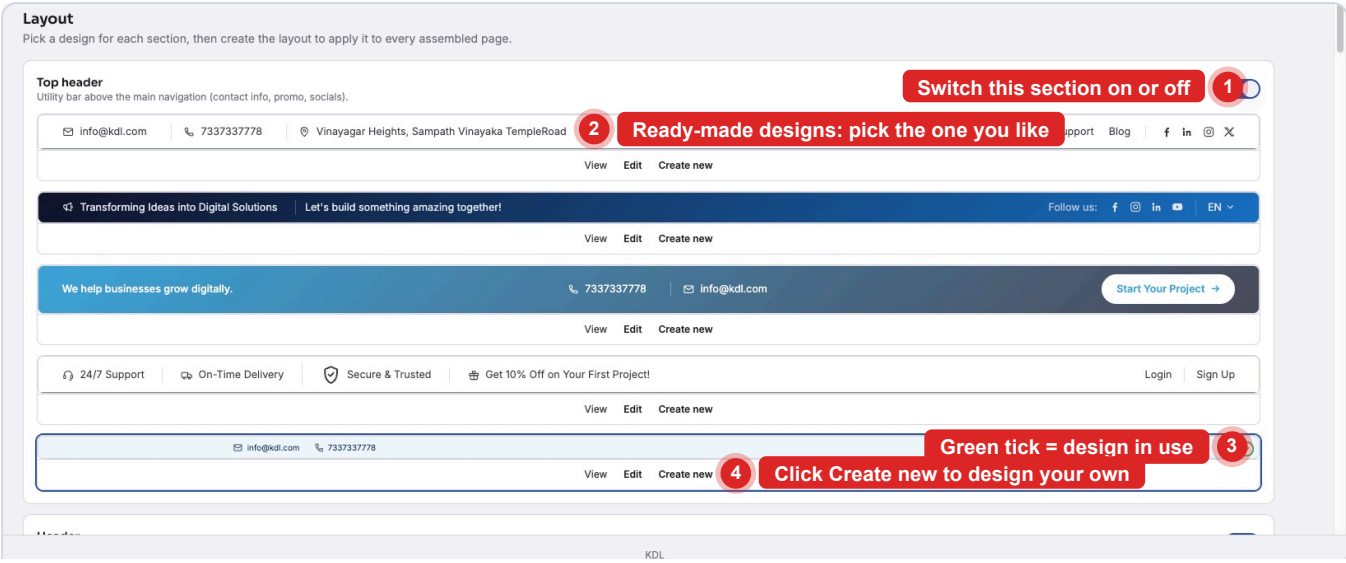
Frontend > Layouts

KDL OS already has ready-made headers and footers. You can use them as they are. If you want a fully custom one, click **Create/Edit layout**.



Layout settings

The **Layout** page lists each section (Top header, Header, Footer and more) with several designs. Use the switch on the right to turn a section on or off, and pick a design: the one with the green tick is in use. For each design you can click **View** to preview it, **Edit** to change it, or **Create new** to build your own from scratch. Then create the layout to apply it to every page. The header and footer of **Home** are shared by every public page.

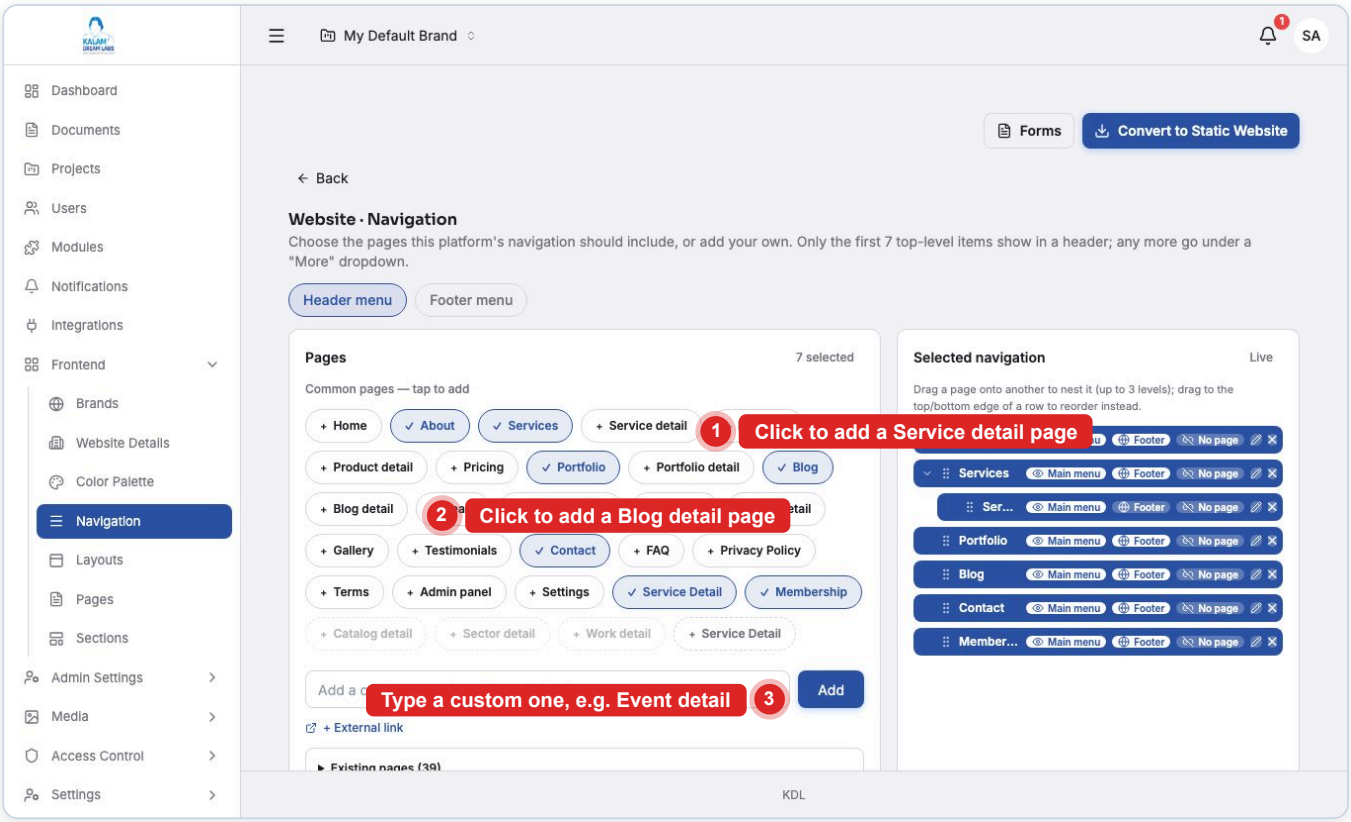


Layout: pick a design for each section

Step 11. Detail pages: one layout for every detail page

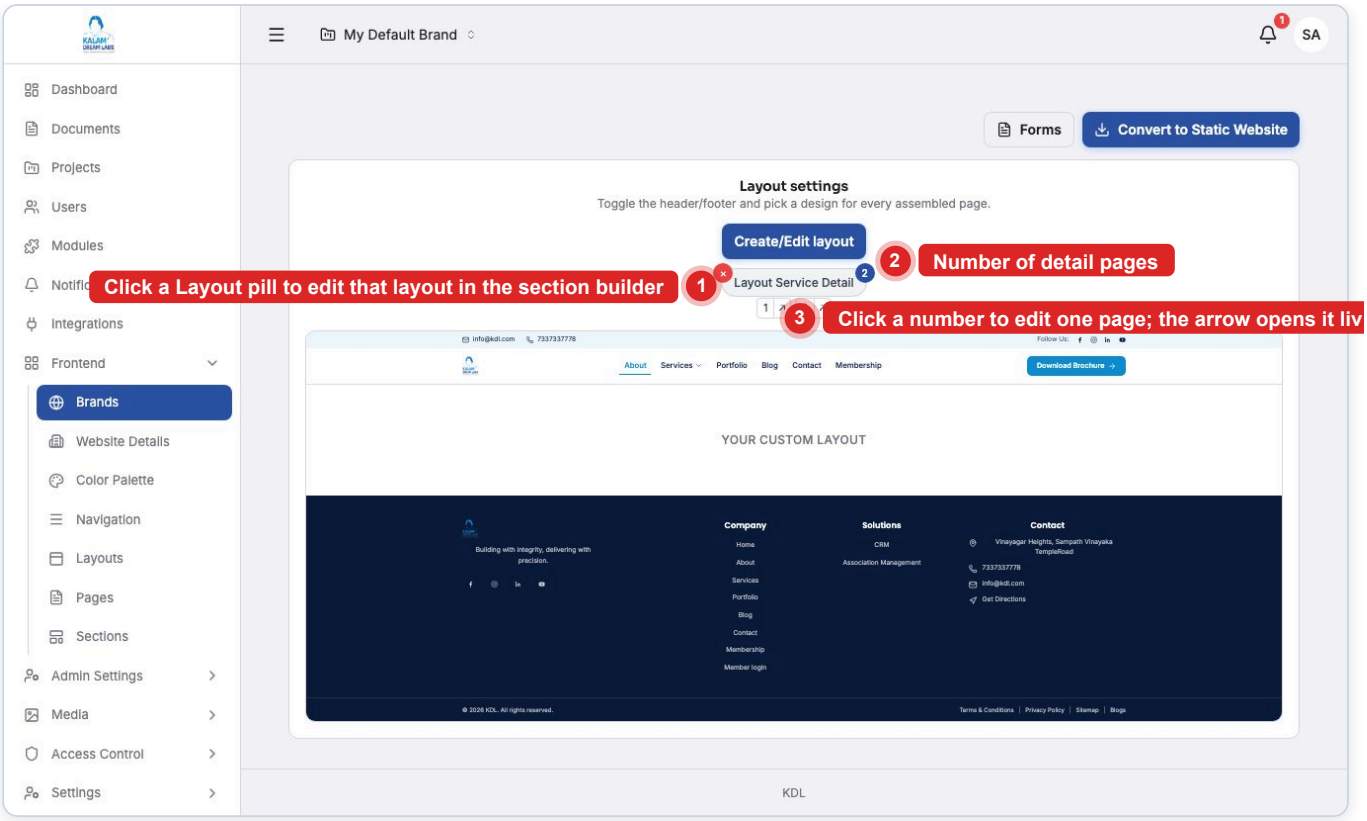
A detail page shows one item: one service, one blog post, one event, one sector. You do not build each by hand. Every item of a type shares one layout, and a page is created for each item.

A. Add the detail page types. Open **Frontend > Navigation**. Click **Service detail**, **Blog detail**, **Portfolio detail**, **Team member** or **Job detail** to add them. For another type, type a name such as **Event detail** and click **Add**. Sector, Catalog and Work detail pages appear once you add entries in those content modules.



Navigation: add the detail pages

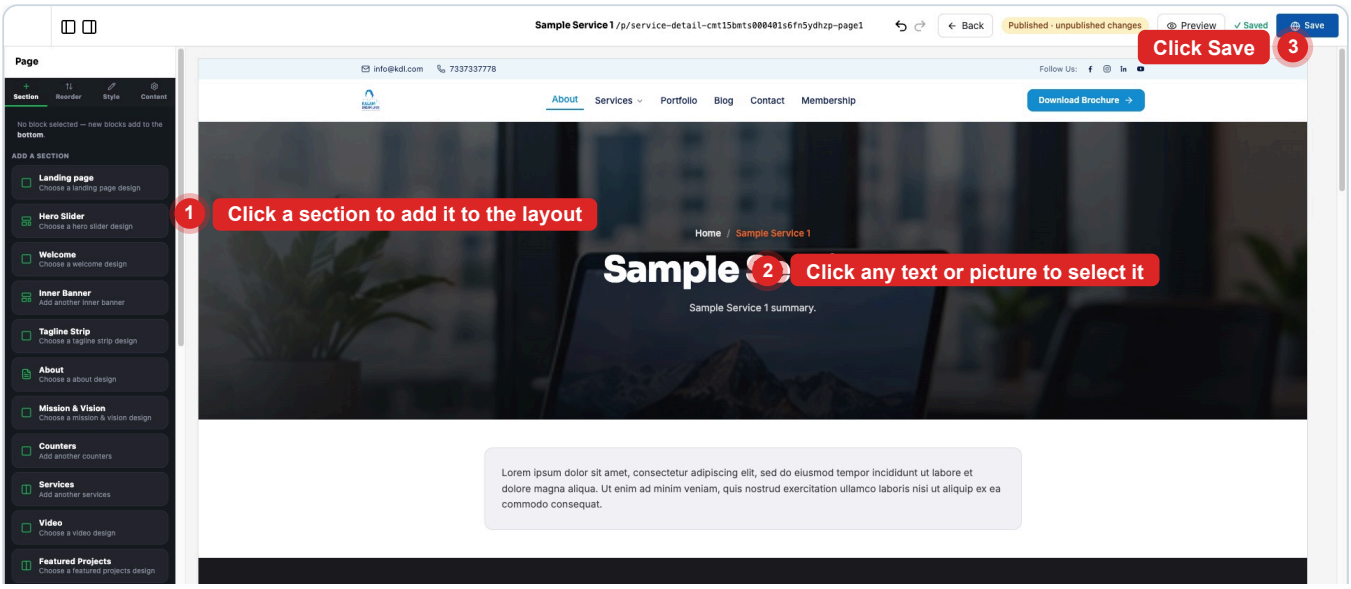
B. Edit the layout. Go back to **Layout settings** (Frontend > Brands > Web app). A pill called **Layout Service Detail** (or Layout Blog Detail, Layout Event Detail and so on) now appears for each type. Click the pill to open the section builder for that layout: change it once and every page of that type changes. The number on the pill is how many detail pages exist. Click a small number under the pill to edit one page's own content (title, text, images); the arrow opens that page live.



Layout settings: the detail page layouts

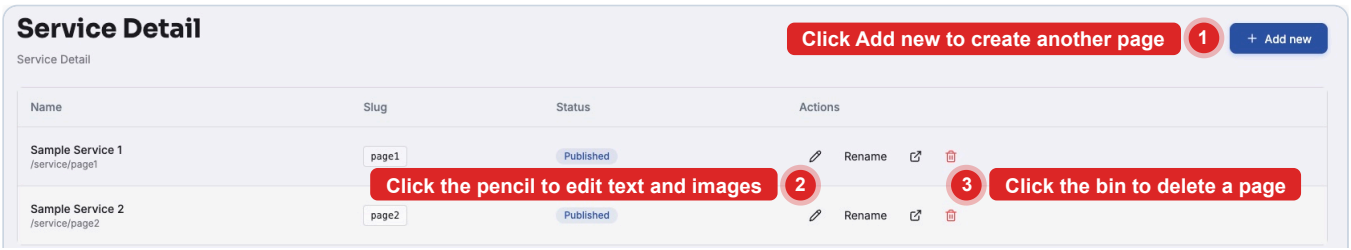
Step 12. Edit the detail layout, then add the content

A. Edit the layout. On Layout settings click the **Layout Service Detail** pill (or the pill of another detail type). The section builder opens. Use the **Section**, **Reorder**, **Style** and **Content** tabs to build the layout, then click **Save**. The layout is the same for every page of this type: you set it once.



Detail page layout in the section builder

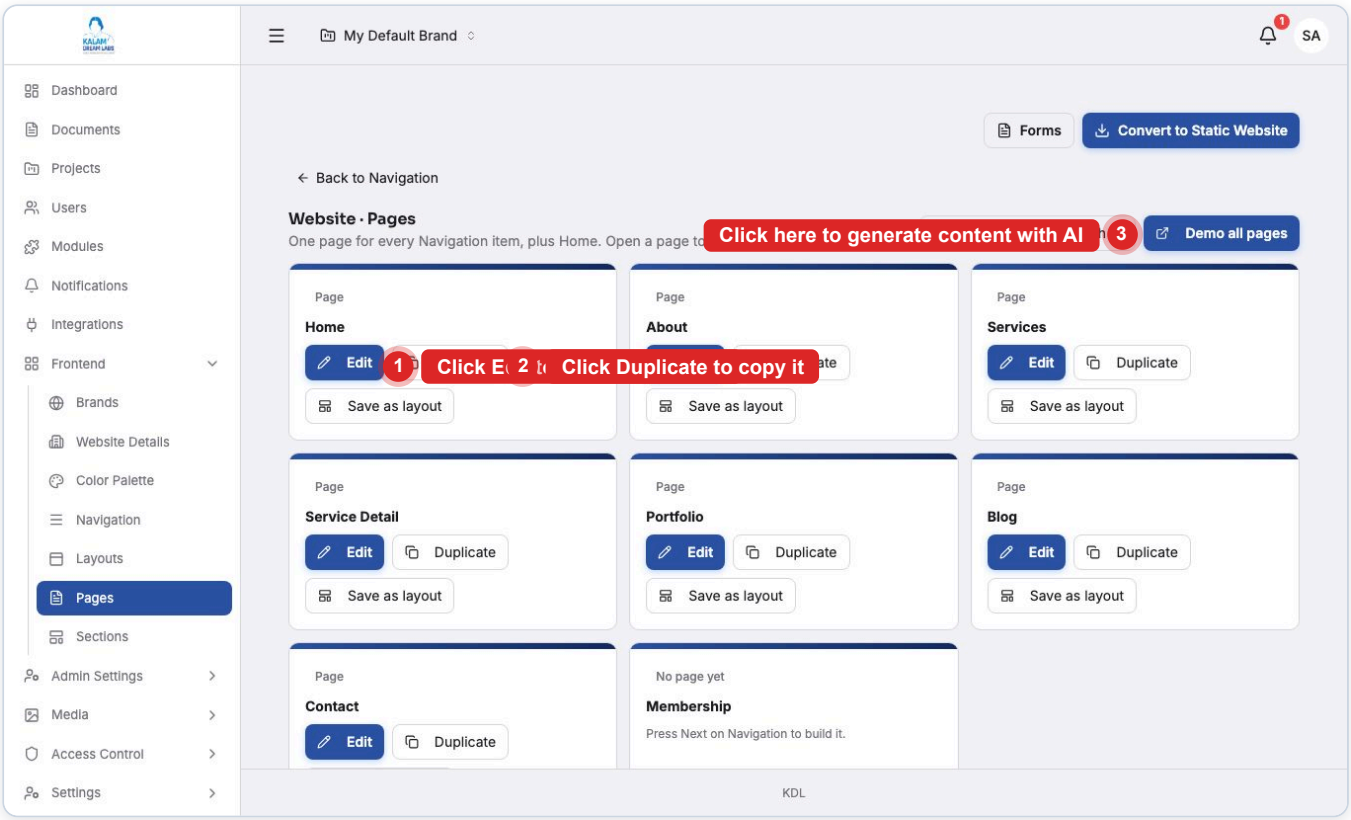
B. Add the content. Go back to Layout settings and click the number on the pill. The list shows every page of this type (for example every service). Click the pencil to change a page's text and images, click **Rename** to rename it, click the arrow icon to open it live, and click the bin to delete it. Click **Add new** to create a related page. The layout never changes between pages: here you change only the text and images.



Service Detail: every page of the type

Step 13. Edit and publish the pages

Open **Frontend > Pages**. Click **Edit** on a page to change its text, images and links in the section builder, then **Save** and **Publish**. Publish stops real problems (an empty page, a missing title, a broken link) and only suggests the rest. Click **Duplicate** to copy a page. **Frontend > Sections** holds reusable sections.



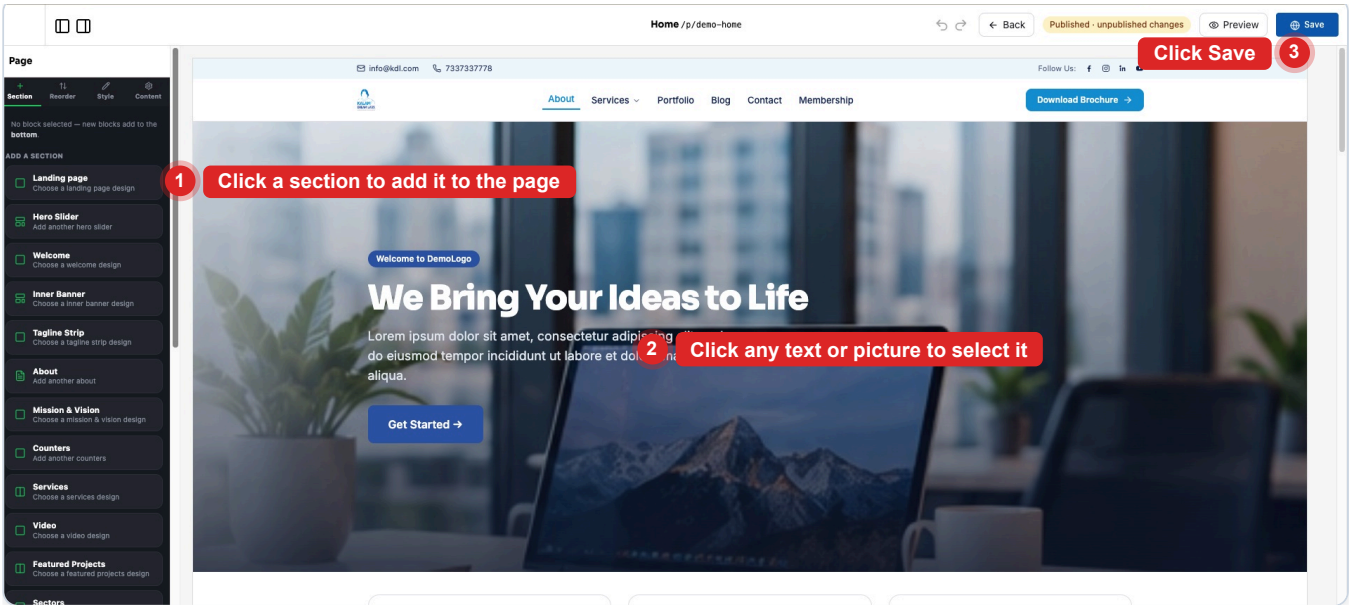
Frontend > Pages

Step 14. Edit the page in the section builder, then Save

In **Frontend > Pages** click **Edit** on a page (for example **Home**). The section builder opens with the page in the middle and the tools on the left:

- **Section:** click a section in the list to add it to the page.
- **Reorder:** move sections up or down.
- **Style:** colours, fonts and spacing.
- **Content:** click a text or a picture on the page to select it, then change its text, image or link here.

When you have finished, click **Save** at the top right. The badge at the top shows the page status.

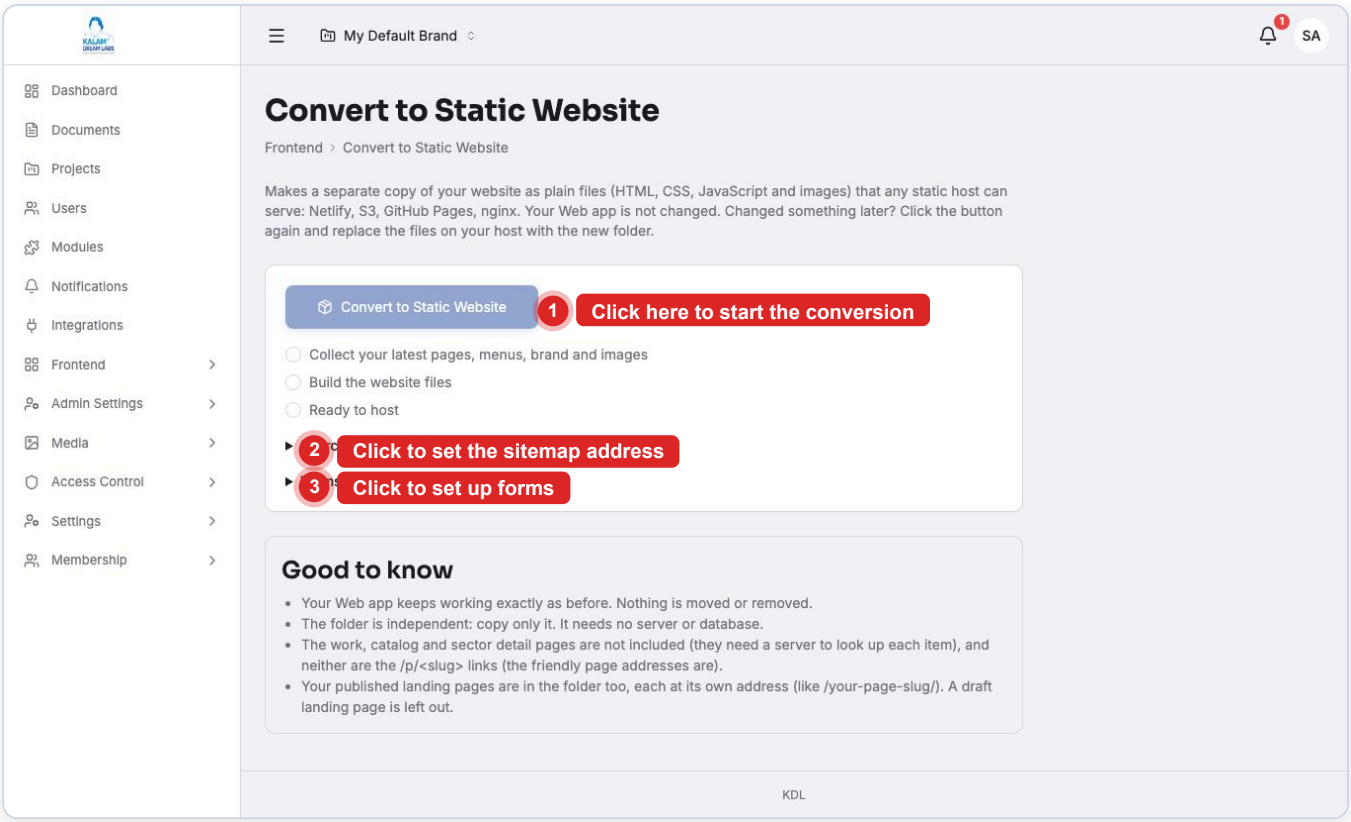


Editing the Home page in the section builder

Step 15. Convert to a static website

Static files (HTML, CSS and images) can be hosted anywhere with no server. Publish your pages first, then click **Convert to Static Website** (the button at the top of the Studio, or http://localhost:3001/admin/static-website).

Optional: type your **Website address** (for example https://www.example.com) to add a sitemap. Type your **Web app address** (https only) if forms must keep working; if you leave it empty, forms show your contact email and phone. Click **Convert to Static Website** and wait a few minutes until the card says it is ready.



Frontend > Convert to Static Website

Step 16. Download the files and host them on GitHub

Click **Download as .zip** on the same page and unzip it. The files inside, with `index.html` at the top, are your finished website.

1. On GitHub create a new empty repository, for example `my-website`.
2. Put the files in a new folder, open a terminal in it, and run (use your own username and repository name):

```
TERMINAL
git init
git add .
git commit -m "Website"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/my-website.git
git push -u origin main
```

3. On GitHub open **Settings > Pages**, choose **Deploy from a branch**, pick **main** and the **/ (root)** folder, then **Save**.
4. After a minute your website is live at `https://YOUR-USERNAME.github.io/my-website/`.

Notes: Changing things later

This is not a step. Use it when something changes.

- **Text, images, links:** Frontend > Pages, Edit, Save, Publish. No re-run is needed.
- **Menus:** Frontend > Navigation.
- **Header, top bar or footer:** Studio > Layouts > Create/Edit layout.
- **Colours or fonts:** Studio > Color Palette, Submit palette.
- After any change: publish, then click **Convert to Static Website** again, replace the files in your GitHub folder, then `git add .`, `git commit -m "Update"` and `git push`. The static copy never updates itself.